`event0 disconnected`，remove 生命周期就已经收敛。反过来，如果只有 `ch6 error` 而没有 Hub 的变化事件，那要查的是 ch7 那条链路。

DMA 缓冲的所有权. 插回键盘时如果连续打印

```
dwc2: control data failed req=0 addr=1 result=-1
```

其中 `req=0, addr=1` 是事件 worker 向根 Hub 发端口 `GET_STATUS`，并不是周期拓扑轮询出错了。这里的根因很隐蔽：早期实现把 4 字节的端口状态数组放在 worker 的内核栈上，又直接把它交给 DWC2 做 DMA。缓存维护按 64 字节 cache line 向外取整，*invalidate* 时会把同一条 cache line 里的其它局部变量一起覆盖掉；而且这样一个短生命周期的栈对象本来就不该成为 DMA 目标。现在端口状态用独立的、静态存续且 64 字节对齐的 `dwc2_hub_port_status[64]`，控制传输仍只接收前 4 字节；Hub 状态位图、端口状态缓冲和枚举用的 `ctrl_buf` 三者彼此分离，DMA 与缓存的归属才不会互相污染。这也是本章反复出现的那条规则的又一次兑现：交给 DMA 的缓冲区必须由传输的 owner 独占，并且活得比传输久。

变化之后的恢复过程是这样的：HCD 先注销那个唯一的发布用 `usb_child`，停止并屏蔽相关 channel、清理待处理中断，然后重新初始化 DWC2 并扫描 Hub。新设备按地址 0 开始枚举，读取 Device Descriptor 前 8 字节确定 EP0 最大包长，设置 USB 地址、读取配置描述符并选择 Boot Keyboard 或 Boot Mouse 接口；随后 USB bus 匹配驱动，执行 probe，重新注册 input 设备并提交对应 HID URB。对低速键盘，地址路由还需记录父 Hub、端口和速度，以便 EP0 与 channel 6 的传输正确使用 split transaction。

与 Linux 还差什么. 把这套驱动和真正的 Linux USB 栈对比，剩下的差距是明确的几条：

- **重枚举的粒度**。Linux 只增删发生变化的 port 和 device；这里的一次拓扑变化会注销旧设备、停止并屏蔽相关 channel，然后重新初始化整个 DWC2。嵌套 Hub 的下游端口变化甚至不会出现在根 Hub 的连接位图里。
- **host channel 的分配**。ch7 固定给 Hub 的 interrupt URB，ch6 固定给唯一那只 HID 数据 URB，因此键盘和鼠标可以拔插、替换，却不能同时长期接收。Linux 是按设备/端点动态调度多个 channel 的。
- **周期调度表**。空闲 NAK 已经用 `retry_work` 加 `dwc2_periodic_wq` 按 `bInterval` 重排，不再占着公共 worker 自旋；但 10 ms 的 delayed-work 粒度只适合这只低速键盘，通用的 HCD 需要以 USB frame/SOF 为不同端点各自维护周期表。
- **硬中断里的忙等**。为了满足 split 的 125 μs microframe 期限，快速路径最多在 IRQ 里等 500 μs 并重启 channel。Linux DWC2 用更完整的 SOF/periodic 状态机把这部分忙等压得更低。
- **多设备发布**。`usb_child` 仍是一个而不是按 Hub 端口排列的数组，所以 USB 键盘与 MT7601U 同时插入时还没有完整发布成两个独立的 `usb_device`，各自保存地址、toggle、host channel 与拔出生命周期。

7.4.8 真机排错：从 STALL 到第一份报告

这个驱动在 QEMU 上很快就能工作，到了真机（低速无线键盘接收器 `2a7a:8a47` 插在 LAN9514 的 port 3）却经历了九个阶段才读到第一个按键。表 7.4 按时间顺序列出每一步的现象、根因和修复；每一步只把串口日志实际证明的东西当作结论——ACK 不等于下游数据已经返回，“ready” 也不等于收到了按键。

其中几个阶段值得多说几句。

阶段 4：“读到了” 但其实没读全. 低速设备端点 0 的最大包长只有 8 字节，18 字节的设备描述符要分 3 个数据包传。经过 split 事务时，每个包都要单独走一遍 SSPLIT/CSPLIT，而且收完一个完整的包后要把 DATA0/DATA1 翻转。旧代码把整个 18 字节当成一次 split 请求，只取回了第一个包，后面 10 字节还是缓冲区里的 0——于是日志说 `vid=0 pid=0`。现在 `channel_xfer()` 对 split 请求按最大包长切片，`get_device_descriptor()` 还检查长度、类型和 VID 不为 0，宁可失败也不接受半截数据。